x calibration and reduced parameters in citkid

This document derives the x calibration (conversion of a complex S21 timestream into the fractional frequency shift x) and the reduced noise parameters, as implemented in `citkid.xcal` and run by `citkid.pipeline`. Every formula below is one the code uses; the function that implements each step is given as `module.function`. Where the code makes a choice that is not explained, or where its behavior differs from its docstring, this is stated.

Contents

1. Inputs and notation
2. Order of steps in the pipeline
3. Gain fit and removal
4. Circle fit
5. Centering and rotation
6. theta and A
7. x of the sweep
8. Choosing the sweep points to fit (xcal mask)
9. The x(theta) polynomial and the x timestream
10. Noise spectra: Sxx, Spar, Sper
11. Reduced parameters
12. Off-resonance and IQ-only templates
13. Sign conventions
14. Assumptions and limitations
15. Worked example: ideal resonator

1. Inputs and notation

Each row (`data_idx`) of a `pipeline.DataSet` is one resonator measured with:

Symbol	Pipeline name	Meaning
f_g, z_g	`fg`, `zg`	gain sweep (frequency in Hz, complex S21)
f_f, z_f	`ff`, `zf`	fine sweep across the resonance
f_t	`ft`	tone frequency of the timestream
z_t	`zt`	complex S21 timestream at f_t
Δt	`dt`	timestream sample time in s

Sweep arrays are sorted by ascending frequency by the loader steps (`load_sweep_f`, `load_sweep_g` in the custom-steps templates); several steps below rely on this.

Throughout, subscript f means "fine sweep" and t means "timestream". arg is `np.angle`, with values in $(-\pi, \pi]$.

Per-row and global quantities. The steps declare their scope in `pipeline/default_steps.py`:

- Global (one value for the whole dataset): `fres_all`, `qres_all`, `dt`, `nrows` (loaded), and `fr_spans` (`make_fr_spans`).
- Global per resonator (`global-res`): `fres`, `qres`, `ares`, `res_idxs`.
- Per row: everything else, i.e. `p_amp`, `p_phase`, `gain_mask`, `circ_origin`, `circ_radius`, `idx_t`, `theta_phase_offset`, `xcal_mask`, `poly_x`, and all derived arrays (`zf_rmv`, `thetaf`, `xt`, `sxx`, ...) and reduced parameters. `ft` and `zt` are per row (loaded with a vectorized loader in the template).

So the gain, circle, rotation, and x polynomial are all fit separately for every row. Because per-row results are stored as one array across rows, each per-row output must have the same shape in every row (Section 9).

2. Order of steps in the pipeline

The analysis steps (fits) are listed in the analysis YAML templates and the calibration steps (transformations) in the calibration YAML templates, both in `citkid/pipeline/templates/`. Calibration outputs are computed on demand when an analysis step needs them (`pipeline.analysis.AnalysisRunner`).

For the on-resonance timestream case (`analysis_ts.yaml` + `cal_ts.yaml`), the order of operations is:

#	Step	Function	Produces
A1	`make_fr_spans`	`xcal.gain.make_fr_spans`	`fr_spans`
A2	`fit_gain`	`xcal.gain.fit_gain`	`p_amp`, `p_phase`, `gain_mask`
C	`rmv_gain_f`	`xcal.gain.remove_gain`	`zf_rmv`
A3	`fit_iq_circle`	`xcal.circle.fit_iq_circle`	`circ_origin`, `circ_radius`
A4	`get_idx_t`	`xcal.xcal.nearest_index`	`idx_t`
C	`rmv_gain_t`	`xcal.gain.remove_gain`	`zt_rmv`
A5	`get_theta_phase_offset`	`xcal.circle.get_theta_phase_offset`	`theta_phase_offset`
C	`center_f`, `get_thetaf`	`xcal.circle.cent_rot_s21`, `convert_to_theta`	`zf_cent`, `thetaf`
C	`center_t`, `get_thetat`	`xcal.circle.cent_rot_s21`, `convert_to_theta`	`zt_cent`, `thetat`
A6	`get_xcal_mask`	`xcal.xcal.get_xcal_mask`	`xcal_mask`
C	`get_xf`	`xcal.xcal.calc_x`	`xf`
A7	`fit_x_theta`	`xcal.xcal.fit_x_theta` (degree 3)	`poly_x`
C	`get_xt`	`np.polyval`	`xt`
C	`get_At`, `get_sparper`	`xcal.circle.convert_to_A`, `get_spar_sper`	`At`, `f_sparper`, `spar`, `sper`
C	`get_sxx`	`signal.psd.get_psd`	`f_sxx`, `sxx`
C	`get_sxx_reduced`, `get_sfactor_reduced`	`xcal.reduced_params.get_*`	`sxx_red`, `sfactor_red` (default `n_avg = 300`)

A = analysis step, C = calibration step. `analysis_ts_fixed_theta.yaml` is identical except that A6 is `get_xcal_mask_fixed_theta` (`xcal.xcal.get_xcal_mask_fixed_theta`). The off-resonance and IQ-only templates are described in Section 12.

3. Gain fit and removal

Resonance spans

`xcal.gain.make_fr_spans` gives, for each resonance in the dataset (`fres_all`, `qres_all`), the pair $(f_{r,i}, f_{r,i}/Q_{r,i})$: the resonance frequency and its full linewidth.

Gain mask

`xcal.gain.get_res_mask` removes the intervals $[f_{r,i} - s_i/2,\ f_{r,i} + s_i/2]$, with $s_i = m\, f_{r,i}/Q_{r,i}$ and $m =$ `span_mult` (default 1), from the gain sweep. Overlapping intervals are merged. `gain_mask` is True on the points that are kept.

Gain model

`xcal.gain.fit_gain` fits the kept gain-sweep points with

$$G_{\mathrm{dB}}(f) = 20 \log_{10} |z_g| \approx P_{\mathrm{amp}}(f) = p_2 f^2 + p_1 f + p_0,$$

$$\phi_g(f) = \mathrm{unwrap}(\arg z_g) \approx P_{\mathrm{phase}}(f) = q_1 f + q_0,$$

with f in Hz (no centering or scaling of f in the call; `np.polyfit` does its own column scaling). The amplitude is fit once, to all kept points.

The phase is unwrapped over the kept points (concatenated across the cut resonances), then fit **separately on each contiguous run** of kept points, "to avoid unwrapping issues" (code comment): a jump across a cut resonance may add a wrong multiple of 2π to a whole run, which changes its intercept but not its slope. Runs with at least 4 points are fit (runs with 2 or 3 points are used only if no run has 4). The fits are then filtered by length: for $n = 101, 91, \ldots, 1$, the first threshold for which some run has more than n points selects those runs, and `p_phase` is the plain mean of their coefficient vectors (q_1, q_0). A wrong $2\pi k$ in one run therefore biases q_0 by $2\pi k / N_{\text{runs}}$; this constant phase is harmless because the later rotation (Section 5) absorbs any constant phase. The slope q_1 (cable delay, $q_1 = -2\pi\tau$ in the convention of `res.funcs.nonlinear_iq`) is what matters.

If the fit fails, `p_amp` and `p_phase` are all NaN (with a warning).

Removal

`xcal.gain.remove_gain` divides out the gain model:

$$z_{\text{rmv}}(f) = \frac{z(f)}{10^{P_{\text{amp}}(f)/20}\, e^{iP_{\text{phase}}(f)}}.$$

It is applied to the fine sweep at each f_f (`zf_rmv`) and to the timestream at the single frequency f_t (`zt_rmv`), using the same `p_amp`, `p_phase`. After removal, off-resonance transmission is ≈ 1 (0 dB, zero phase), and amplitudes are in units of the off-resonance carrier.

4. Circle fit

`xcal.circle.fit_iq_circle` fits a circle with center $c = A + iB$ and radius R to `zf_rmv` (restricted to `circ_mask`; the templates use `circ_mask: null`, i.e. the whole fine sweep). It minimizes the algebraic objective (`xcal.circle.circle_objective`)

$$\chi^2(A, B, R) = \sum_k \left[(I_k - A)^2 + (Q_k - B)^2 - R^2\right]^2, \qquad I_k + iQ_k = z_{\text{rmv},k},$$

with `scipy.optimize.fmin` (Nelder-Mead), starting from the center and mean half-width of the bounding box. This is not the geometric (orthogonal distance) fit, and all points are weighted equally. `circ_origin` $= c$ and `circ_radius` $= R$.

5. Centering and rotation

`xcal.circle.cent_rot_s21` maps gain-removed data to circle-centered, rotated coordinates:

$$z_{\text{cent}} = (z_{\text{rmv}} - c)\, e^{-i\varphi_0},$$

where φ_0 = `theta_phase_offset`. The same c and φ_0 are used for the sweep (`center_f`) and the timestream (`center_t`).

Phase offset

`xcal.circle.get_theta_phase_offset` sets

$$\varphi_0 = \arg\left[\text{median}\,\text{Re}(z_{\text{rmv},t} - c) + i\,\text{median}\,\text{Im}(z_{\text{rmv},t} - c)\right],$$

so that the timestream sits near $\theta = 0$ after rotation. The medians of the real and imaginary parts are taken separately (`np.median` of a complex array would sort by the real part). For a compact noise blob this is close to the median of θ_t, so the rotated θ_t is centered on zero. Any residual offset would not bias x_t anyway, because the sweep is rotated by the same φ_0 and the x polynomial is fit in that common frame.

Because φ_0 comes from the timestream, it is a per-row property of the (sweep, timestream) pair, and the same rotation must be applied to both.

6. theta and A

`xcal.circle.convert_to_theta` and `xcal.circle.convert_to_A` give

$$\theta = \arg z_{\text{cent}}, \qquad A = |z_{\text{cent}}|.$$

θ is the angle around the circle center (radians), measured from the direction of the timestream; A is the distance from the center, in units of the off-resonance carrier (so $A \approx R$ on the circle).

Unwrapping and `idx_t`

- **Timestream** (`get_thetat`): not unwrapped. $\theta_t \in (-\pi, \pi]$, and since the timestream is rotated to $\theta \approx 0$, it is continuous as long as the timestream stays within $\pm\pi$ of its median.
- **Fine sweep** (`get_thetaf`): unwrapped along the (frequency-sorted) sweep with `np.unwrap`, which is only correct if consecutive sweep points differ by less than π in angle. `np.unwrap` starts from the first point, so the result can be offset from the timestream branch by $2\pi k$. To fix the branch, `idx_t = xcal.xcal.nearest_index(ff, ft)` (the sweep index closest in frequency to f_t) is used:

  $$k = \text{round}\left(\frac{\theta_f[\text{idx}_t]}{2\pi}\right), \qquad \theta_f \leftarrow \theta_f - 2\pi k,$$

  so $\theta_f[\text{idx}_t] \in [-\pi, \pi]$, the same branch as θ_t. This assumes that the sweep point nearest the tone is close in θ to the timestream (i.e. the resonance did not move far between the sweep and the timestream). If $\theta_f[\text{idx}_t]$ is near $\pm\pi$, the branch choice is ambiguous.

7. x of the sweep

`xcal.xcal.calc_x` (step `get_xf`) assigns each fine-sweep point

$$x_f = 1 - \frac{f_f}{f_t}.$$

Why: the resonator response depends on frequency through $y = Q_r(f - f_r)/f_r$. A tone at fixed f_t on a resonator shifted to $f_r = f_{r0}(1 + \delta)$ gives the same S21 as a sweep point f on the unshifted resonator when $f/f_{r0} = f_t/f_r$, i.e. $f = f_t/(1 + \delta) \approx f_t(1 - \delta)$. So, to first order,

$$x_f \approx \delta = \frac{f_r - f_{r0}}{f_{r0}},$$

the fractional resonance frequency shift that would move the timestream to the state seen at sweep point f_f, referenced to the state at the tone ($x = 0$ at $f_f = f_t$). The code uses $1 - f/f_t$, not the exact $f_t/f - 1$; the two agree to first order in x (the difference is $\mathcal{O}(x^2)$, about 10^{-8} for $x \sim 10^{-4}$).

8. Choosing the sweep points to fit (xcal mask)

Two analysis steps make `xcal_mask`, the subset of fine-sweep points used for the x(theta) fit.

Timestream range: `xcal.xcal.get_xcal_mask`

Arguments `idx0_offset`, `idx1_offset`, `std_cutoff` (function defaults 3, 7, 12; `analysis_ts.yaml` uses 3, 9, 16).

1. Glitch cut (`xcal.xcal.get_theta_t_bulk_mask`): keep timestream points with $|\theta_t - \overline{\theta_t}| < n_\sigma \,\text{std}(\theta_t)$, n_σ = `std_cutoff` (None keeps all). Let θ_{min}, θ_{max} be the min and max of the kept θ_t.
2. Collect sweep indices j with $\theta_{\text{min}} \le \theta_f[j] \le \theta_{\text{max}}$, plus the indices $j + 1$ where θ_f crosses θ_{min} or θ_{max} between j and $j + 1$ (so a timestream range that falls between two sweep points is still bracketed).

3. $\text{idx}_0 = \min(\text{indices}) - 1$, $\text{idx}_1 = \max(\text{indices})$, then widen by o_0 = `idx0_offset` and o_1 = `idx1_offset`: $\text{idx}_0 \leftarrow \max(0, \text{idx}_0 - o_0)$, $\text{idx}_1 \leftarrow \min(N-1, \text{idx}_1 + o_1)$.
4. The mask is the contiguous index range $[\text{idx}_0, \text{idx}_1]$.

Indices are in ascending-frequency order, so `idx0_offset` extends the fit toward lower frequency (larger θ for the usual loop orientation, Section 13) and `idx1_offset` toward higher frequency. The code does not document why the defaults are asymmetric. If no sweep point is in or brackets the timestream range, `min` of the empty index list raises a `ValueError`.

Fixed theta band: `xcal.xcal.get_xcal_mask_fixed_theta`

Ignores the timestream and keeps sweep points with

$$\min \theta_f + \Delta_{\text{min}} \le \theta_f \le \max \theta_f - \Delta_{\text{max}},$$

$\Delta_{\text{min}}, \Delta_{\text{max}}$ = `xcal_theta_min_offset`, `xcal_theta_max_offset` (0.2 rad each by default and in `analysis_ts_fixed_theta.yaml`). NaNs are never kept, and the mask need not be contiguous. It raises if no point is in the band. This covers most of the loop, so it is useful when the timestream range is large or unreliable, at the cost of fitting a wide range of θ with a cubic (Section 14).

9. The x(theta) polynomial and the x timestream

`xcal.xcal.fit_x_theta` (step `fit_x_theta`) fits

$$x_f \approx P_x(\theta_f) = \sum_{n=0}^{3} a_n \theta_f^n$$

to the masked sweep points with `np.polyfit` (unweighted least squares in x). `poly_x` $= (a_3, a_2, a_1, a_0)$ in `np.polyval` order. (The function's first argument is named `thetat` and documented as "theta values from the noise measurement", but the pipeline passes `thetaf`, the sweep theta, which is what is fit.)

The degree is fixed to 3 in `pipeline/default_steps.py` on purpose: `poly_x` is a per-row output stored as one array across rows, so it must have the same shape (4 coefficients) in every row. A different degree needs a custom step.

The timestream is then calibrated by step `get_xt`:

$$x_t = P_x(\theta_t).$$

This uses only the angle; the radial coordinate A_t is not used for x. The calibration maps each timestream sample to the sweep state with the same angle around the circle center, i.e. it assumes the timestream moves along the sweep's IQ loop (Section 14).

The per-row calibration can be written compactly as $z_{\text{cent}} = z_t z_1 + z_2$ with $z_1 = e^{-i\varphi_0}/(10^{P_{\text{amp}}(f_t)/20} e^{iP_{\text{phase}}(f_t)})$, $z_2 = -c\, e^{-i\varphi_0}$, then $x_t = P_x(\arg z_{\text{cent}})$. This is the same transformation as the pipeline steps above, which apply it one step at a time.

10. Noise spectra: Sxx, Spar, Sper

All PSDs use `signal.psd.get_psd`, the one-sided periodogram of the whole timestream with no window, no averaging, and no mean subtraction:

$$S(f_k) = \frac{2\,\Delta t}{N} \left| \sum_{n=0}^{N-1} s_n e^{-2\pi i k n/N} \right|^2, \qquad f_k = \frac{k}{N\Delta t}, \quad k = 0, \ldots, \lfloor N/2 \rfloor.$$

The factor 2 is applied to every bin, including DC (and Nyquist for even N). The reduced parameters exclude $f = 0$.

- **Sxx** (`get_sxx`): S_{xx} is `get_psd` applied to x_t, in 1/Hz.

- **Spar, Sper** (`xcal.circle.get_spar_sper`, step `get_sparper`):

$$S_{\parallel} = 10\log_{10}\mathrm{PSD}(R\,\theta_t), \qquad S_{\perp} = 10\log_{10}\mathrm{PSD}(A_t),$$

  in dB relative to the off-resonance carrier per Hz (dBc/Hz), since the gain was removed. $R\theta_t$ is the arc-length displacement along the circle (tangential, "parallel" to the loop), and A_t the distance from the center (radial, "perpendicular" to the loop). The mean of A_t ($\approx R$) only enters the DC bin. R is `circ_radius`.

For small excursions, the tangential direction is the direction the state moves when the resonance frequency shifts, and the radial direction is (to first order at the operating point) the direction of a change in dissipation; this is the usual reading of $S_{\parallel}$ and $S_{\perp}$. The code itself only computes the two projections.

11. Reduced parameters

`xcal.reduced_params` reduces each spectrum to a few numbers at the default frequencies $\nu \in \{0.1, 0.3, 1, 3, 10, 30, 100, 300\}$ Hz (`reduced_params._freqs`). For each ν, the band is the open interval $0.8\nu < f < 1.2\nu$:

- `get_sxx_reduced`: linear mean of S_{xx} in the band, $\langle S_{xx}\rangle_\nu$ (1/Hz).
- `get_sfactor`: mean of $S_{\parallel} - S_{\perp}$ in the band, in dB:

$$\mathrm{sfactor}_\nu = \left\langle 10\log_{10}\frac{\mathrm{PSD}(R\theta_t)}{\mathrm{PSD}(A_t)} \right\rangle_\nu,$$

  i.e. the mean of the dB ratio (a log average), not the ratio of the band means.

If 0.8ν is below the lowest positive frequency $1/(N\Delta t)$, or 1.2ν is above the highest frequency, the single-frequency functions raise and the `*_default_freqs` wrappers return NaN. For example, the 0.1 Hz values need a timestream of at least 12.5 s, and the 300 Hz values need $f_s/2 \geq 360$ Hz.

Pipeline names: the wrappers return dicts keyed `sxx_0p1`, `sxx_0p3`, `sxx_1`, ..., `sxx_300` and `sfactor_0p1`, ..., `sfactor_300` (`.` replaced by `p`, see `reduced_params.default_freq_keys`), and `pipeline/default_steps.py` stores them under the same names.

Meaning:

- $\langle S_{xx}\rangle_\nu$ is a coarse summary of the frequency noise spectrum, useful for flagging rows (e.g. excess low-frequency noise). Since each periodogram bin has 100% scatter, the scatter of the band mean is about $1/\sqrt{N_{\mathrm{bins}}}$, with $N_{\mathrm{bins}} \approx 0.4\nu N\Delta t$; at 0.1 Hz with a 100 s timestream that is about 4 bins (50% scatter).
- $\mathrm{sfactor}_\nu \approx 0$ dB when the noise is the same in both directions, as for noise added after the resonator (e.g. amplifier noise). Positive values mean excess noise along the loop, i.e. noise that looks like resonance frequency fluctuations (or real signal). It is used to flag rows whose S_{xx} is limited by the readout rather than the resonator (e.g. `sfactor_10 < 3` in `notebooks/ts_iq_analysis_minimal.ipynb`). Because each bin's dB value has a biased, non-Gaussian scatter, the log average has some scatter too; the differences of two log-periodograms with equal true PSDs average to about 0 dB.

12. Off-resonance and IQ-only templates

- **Off-resonance** (`analysis_ts_offres.yaml` + `cal_ts_offres.yaml`): no circle fit. `get_theta_phase_offset_offres` uses center $c = 0$, so φ_0 is the angle of the median of $z_{\mathrm{rmv},t}$ (real and imaginary parts separately) and `rotate_t_offres/rotate_f_offres` give $z_{\mathrm{cent}} = z_{\mathrm{rmv}}e^{-i\varphi_0}$. Then θ is the phase of the gain-removed transmission relative to its median and A its amplitude. There is no x calibration. Note: `get_sparper` takes `circ_radius`, which neither off-resonance template produces, so $S_{\parallel}$/sfactor are only available if `circ_radius` is supplied some other way.
- **IQ only** (`analysis_iq.yaml` + `cal_iq.yaml`): gain fit, then a resonator fit (`fit_iq`) to `zf_rmv`. The calibration YAML lists `center_f`, `get_thetaf`, and `get_xf`, but the IQ-only analysis and loader template do not produce `circ_origin`, `theta_phase_offset`, `idx_t`, or `ft`, so those steps cannot run without extra inputs.

13. Sign conventions

- $x = 1 - f/f_t$: a sweep point **below** the tone has $x > 0$. To first order, x is the fractional resonance shift $(f_r - f_{r0})/f_{r0}$, so $x > 0$ means the resonance moved **up**, and an increase in quasiparticle density (which lowers f_r) gives $x < 0$.
- θ is measured counterclockwise in the I-Q plane (`np.angle`), around the circle center, with $\theta \approx 0$ at the timestream.
- Loop direction: with the gain removed, citkid's model (`res.funcs.linear_iq_nogain`, $1 - \frac{(Q_r/Q_c)e^{i\phi}/\cos\phi}{1+2iQ_r(f-f_r)/f_r}$) traverses the circle clockwise as f increases, so θ_f decreases with frequency and **increases with** x: $dx/d\theta > 0$ (Section 15). The x calibration does not assume this sign; `fit_x_theta` fits whatever relation the data have. A measured loop with the opposite orientation (e.g. a conjugated IQ convention in the readout) gives $dx/d\theta < 0$ and is still calibrated correctly.
- `theta_phase_offset` is subtracted ($e^{-i\varphi_0}$); the gain phase model is divided out ($e^{-iP_{\text{phase}}}$).

14. Assumptions and limitations

- **Small signal / quasi-static.** $x_t = P_x(\theta_t)$ assumes the timestream moves along the same IQ loop as the sweep, so that θ alone determines x. Changes in dissipation (loop radius), and dynamics faster than the resonator ring-down, are not modeled. Nonlinear (bifurcating) resonators distort the loop away from a circle and make the sweep (taken with a frequency-dependent drive history) and the timestream inconsistent.
- **Circle model.** The center and radius come from an unweighted algebraic circle fit to the whole fine sweep (by default). Loops that are not circular (nonlinearity, nearby resonances, residual gain slope) bias c, and therefore θ.
- **Gain model.** Amplitude is quadratic in f (dB) and phase linear in f over the gain sweep, with resonances cut by $\pm$ half a linewidth (times `span_mult`). Wings of the resonances outside the cut, or gain structure not captured by these polynomials, enter `zf_rmv` and `zt_rmv`. The timestream gain correction is the model evaluated at the single frequency f_t.
- **Polynomial degree 3** is fixed for storage-shape reasons (Section 9), not because it is optimal. Over a narrow timestream range a cubic is very accurate; over the wide range of the fixed-theta mask, the ideal $x(\theta)$ has the shape of $\tan(\theta/2)$ (Section 15), which a cubic cannot follow near $\theta \to \pm\pi$.
- **Unweighted x fit.** All masked sweep points carry equal weight, and the fit minimizes residuals in x (frequency is treated as exact; noise in θ_f is ignored).
- **Branch choice** relies on the sweep point nearest f_t being on the same 2π branch as the timestream (Section 6).
- **First-order x.** $x = 1 - f/f_t$ equals the fractional shift only to first order (Section 7).
- **Phase offset** uses the separate medians of the real and imaginary parts, which approximates the angular median for a compact noise blob (Section 5); any residual offset does not affect x_t.
- **Spectra** are single unwindowed periodograms of the full timestream; the reduced parameters average a few bins (Section 11).

15. Worked example: ideal resonator

This illustrates the calibration; the code does not use these formulas.

For `res.funcs.linear_iq_nogain` with $\phi = 0$, the circle has center $c = 1 - Q_r/(2Q_c)$ and radius $R = Q_r/(2Q_c)$, and

$$z - c = \frac{Q_r}{2Q_c}\,\frac{2iy-1}{1+2iy} = -R\,e^{-2i\arctan(2y)}, \qquad y = Q_r\frac{f-f_r}{f_r}.$$

With the tone on resonance ($f_t = f_r$), the rotation by $\varphi_0 = \pi$ puts the tone at $\theta = 0$, and

$$\theta = -2\arctan(2y).$$

For a sweep point $f = f_t(1-x)$, $y = -Q_r x$, so $\theta = 2\arctan(2Q_r x)$ and

$$x(\theta) = \frac{\tan(\theta/2)}{2Q_r} = \frac{\theta}{4Q_r} + \frac{\theta^3}{48Q_r} + \mathcal{O}(\theta^5).$$

The linear coefficient $dx/d\theta = 1/(4Q_r)$ is the responsivity, and the cubic captures the leading nonlinearity. Running `fit_iq_circle`, `get_theta_phase_offset`, `convert_to_theta`, `get_xcal_mask`, and `fit_x_theta` on a simulated $Q_r = 2 \times 10^4$ loop with a noisy on-resonance timestream gives $a_1 = 1.2498 \times 10^{-5}$ $(1/(4Q_r) = 1.25 \times 10^{-5})$ and $a_3 = 1.09 \times 10^{-6}$ $(1/(48Q_r) = 1.04 \times 10^{-6})$.